

Designing Scalable Applications for Millions of Records

1. Architect Mindset

Understand system type (read-heavy/write-heavy), expected load, data size, and consistency requirements.

2. High-Level Architecture

Client → API Gateway → Load Balancer → Microservices → DB + Cache + Queue

3. Handling Millions of Records

Use database indexing, partitioning, and sharding. Introduce caching with Redis and async processing using Kafka.

4. Application Design

Follow clean architecture (Controller → Service → Repository) and use design patterns like Strategy and Factory.

5. Concurrency

```
ExecutorService executor = Executors.newFixedThreadPool(10);
```

6. Deployment

Use AWS, Docker for containerization, and Kubernetes for orchestration.

7. Monitoring

Use Prometheus and Grafana for monitoring, ELK stack for logging.

8. CI/CD

Code → Build → Test → Docker → Deploy

9. Failure Handling

Implement retries, circuit breakers, and Dead Letter Queues.

10. Example Flow

1. User places order
2. API receives request
3. Publish event to Kafka
4. Async processing
5. Store result in DB
6. Cache using Redis